

CampusEats — External SOAP Partner Integration

CS 543 Web Services · Assignment 3 · Design-time contract package

Team CampusEats 26 August 2026

Team ID: CampusEats

Members: Mohit Tiwari (20252651034) · Himanshi Pawar (20252651024) · Ajay Kumar (20252651004) · Asha Sahu (20252651013) · Vedant Sahu (20252651063)

Context

CampusEats integrates the external **PaySecure Payment Gateway** through one operation, `chargePayment`, called by the CampusEats Payment Service when the Order Service requests payment authorization. This edge uses SOAP because a payment partner benefits from a strict, machine-readable WSDL contract and predictable typed faults. SOAP headers also provide a defined place for credentials and can later be extended to WS-Security signatures without changing the business payload. The rest of CampusEats remains REST because browsing, ordering, delivery tracking, and reviews are resource-oriented interactions that benefit from lightweight JSON and ordinary HTTP semantics; the payment edge additionally needs idempotent processing and transaction-style certainty.

The submitted `partner.wsdl` is the partner contract to which the CampusEats Payment Service binds. Its single document/literal SOAP 1.1 operation accepts the CampusEats order reference, amount, currency, opaque payment token, and idempotency key; it returns the gateway transaction reference and authorization result.

HTTP binding

The WSDL binds `chargePayment` to SOAP 1.1 over HTTP. The endpoint and `SOAPAction` below are copied exactly from the WSDL service/port and binding.

```
POST /soap/v1/payments HTTP/1.1
Host: api.paysecure.example
Content-Type: text/xml; charset=utf-8
SOAPAction: "https://api.paysecure.example/soap/v1/chargePayment"
Content-Length: <calculated UTF-8 byte length>

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:pay="https://schemas.paysecure.example/v1">
  <!-- Header and Body are exactly those in soap-request.xml. -->
</soapenv:Envelope>
```

Endpoint URL: `https://api.paysecure.example/soap/v1/payments`

Discovery

At design time, CampusEats would query the university integration catalogue for an approved payment service, select the active PaySecure record, and download the immutable WSDL from the record's contract URL. The catalogue supplies governance metadata and a tModel-style contract pointer; it is not a live UDDI server. A direct partner URL may be used after selection, but builds should pin a reviewed WSDL version in the CampusEats repository so an upstream change cannot silently alter generated client bindings.

Catalogue / registry record

Field	Registered value
Business	PaySecure Technologies Pvt. Ltd. (<code>businessKey: ps-business-001</code>)
Service	PaySecure Gateway v1 — Card Authorization (<code>serviceKey: ps-charge-v1</code>)
Endpoint	<code>https://api.paysecure.example/soap/v1/payments</code>
Binding	SOAP 1.1, document/literal, HTTPS
WSDL / tModel pointer	<code>https://developer.paysecure.example/wsd1/v1/paysecure-gateway.wsd1</code> (<code>tModelKey: tmodel:paysecure:gateway:v1</code>)

Fault mapping

The adapter inside the CampusEats Payment Service owns the translation boundary. When `soap-fault.xml` carries partner detail code `CARD_DECLINED`, the adapter records the gateway reference for operations, marks the CampusEats payment as `DECLINED`, and returns the Assignment 2-facing error **`PAYMENT_DECLINED` — “Payment was declined. Use another payment method.”** to the Order Service's `authorizePayment` call. The Order Service then exposes its own `placeOrder` failure with the same CampusEats wording; neither `CARD_DECLINED`, PaySecure's fault string, nor the issuing-bank message is shown to students.

Transport failure or a retryable gateway fault would instead map to `PAYMENT_UNAVAILABLE`, allowing a safe retry with the same idempotency key. This keeps partner vocabulary and sensitive diagnostics behind the Payment Service contract.

Contract consistency checklist

- `partner.wsd1` contains `types`, four `message` declarations, `portType`, SOAP-over-HTTP `binding`, and `service/port` with an HTTPS endpoint.
- `soap-request.xml` places credentials in the SOAP Header and the operation payload in the Body.

- `soap-response.xml` and `soap-fault.xml` use the same namespace, wrapper elements, element order, and datatypes declared by the WSDL.
- The HTTP request uses the binding's exact `SOAPAction` and the service/port endpoint.
- Discovery is represented by one modern catalogue record, with a tModel-style WSDL pointer and no running UDDI dependency.